CUDA-ACCELERATED IMAGE PROCESSING WITH KERNEL OPTIMIZATION AND PERFORMANCE BENCHMARKING

Yashvardhan Gupta,55,230905442

Shubhendu Arya,56,230905458

Abhiyuday Verma,52,230905418

Department of Computer Science and Engineering, MIT Manipal

Guide: Dr. Jyothi Upadhya K | Date: April 2026

Abstract—This report presents a CUDA-based image processing mini-project implementing Gaussian blur, sharpening, Sobel edge detection, dilation, and erosion. Each operation is implemented with CPU baseline, CUDA naive kernel, and CUDA optimized shared-memory kernel variants. A Python Tkinter interface was used for benchmarking and visualization. Experimental results show significant acceleration for optimized kernels, with up to 65.45x speedup over CPU on tested scenarios.

Keywords—CUDA, image processing, GPU acceleration, shared memory, benchmark, Sobel, Gaussian blur, morphology

1. Introduction

Image processing workloads involve repetitive neighborhood operations that are computationally expensive on CPUs for high-resolution images. CUDA enables massive parallel execution by distributing per-pixel computations across thousands of GPU threads. This project develops and evaluates CUDA-accelerated image processing pipelines and compares them against CPU implementations.

The implementation and source code for this project are available in a public GitHub repository .https://github.com/yashvardhancse/pcap-cuda-image-processing-mini-project/

2. Literature Review

Foundational image processing texts identify convolution and morphology as core primitives for enhancement and feature extraction. CUDA literature emphasizes memory hierarchy-aware optimization, especially shared-memory tiling for stencil-style operations. This project follows these principles by implementing naive and optimized kernels and comparing performance impact empirically.

3. Problem Statement

Sequential CPU implementations of blur, edge detection, and morphological operations scale poorly with image size. Even GPU implementations can underperform if kernels are not optimized for memory access patterns. The project addresses this by designing optimized kernels and measuring end-to-end performance improvements.

4. Objectives

- Implement Gaussian blur, sharpening, Sobel edge detection, dilation, and erosion.
- Develop CPU, CUDA naive, and CUDA optimized kernel variants.
- Use appropriate launch configurations with block and grid decomposition.
- Apply shared-memory tiling to reduce global memory traffic.
- Benchmark runtime and compute speedup for practical input images.
- Document implementation and analysis in IEEE-style report format.

5. Methodology

The project uses RGBA images (4 channels per pixel) and processes each pixel in parallel on the GPU. Common helper utilities in common.cuh manage pixel indexing, clamping, device memory allocation, host-device transfer, and launch dimensions. Optimized kernels load a tile plus halo into shared memory and then compute neighborhood operations with low-latency accesses.

5.1 System Architecture

The system is designed as a multi-layer architecture integrating a Python-based user interface with CUDA-accelerated native processing.

- UI Layer: A Tkinter-based desktop application is used to load images, select operations, adjust parameters, and visualize outputs and performance metrics.

- Benchmark Layer: This layer interfaces Python with CUDA using ctypes. It converts images between PIL and NumPy formats and executes three modes of processing: CPU, CUDA naive, and CUDA optimized.

- CUDA Layer: The core image processing operations are implemented in CUDA C++ as GPU kernels. These include Gaussian blur, sharpening, Sobel edge detection, dilation, and erosion. Both naive and shared-memory optimized kernels are implemented.

- Build Layer: The system is compiled into a shared library using CMake and linked with the CUDA runtime library, enabling seamless integration with Python.

5.2 Runtime Data Flow

The execution flow of the system is as follows:

1. The application initializes and loads the CUDA shared library.

2. The user loads an image, which is converted into a NumPy array in RGBA format.

3. Upon selecting an operation and running the benchmark, the system executes three modes: CPU, CUDA naive, and CUDA optimized.

4. For CUDA execution:

- Device memory is allocated using cudaMalloc.
- Input image data is transferred from host to device using cudaMemcpy.
- CUDA kernels are launched with appropriate grid and block configurations.
- Processed data is copied back to host memory.
- Device memory is freed using cudaFree.

5. The results, including processed images and execution times, are displayed in the UI.

5.3 Image Representation

The system processes images in RGBA format, where each pixel consists of four channels: Red, Green, Blue, and Alpha.

All image processing operations are applied to the RGB channels, while the alpha channel is preserved to maintain transparency information. This ensures compatibility with standard image formats and avoids unintended distortion of transparency.

CUDA kernels are launched using a tile-based configuration. Each block consists of 16×16 threads, and the grid is sized to cover the entire image. This configuration ensures efficient utilization of GPU resources and balanced workload distribution across threads.

6. Experimental Setup

Software stack: Python UI (Tkinter), CUDA runtime, C++ kernels, and CMake build pipeline. Benchmark mode measures CPU, CUDA naive, and CUDA optimized execution times for selected operations and input images.

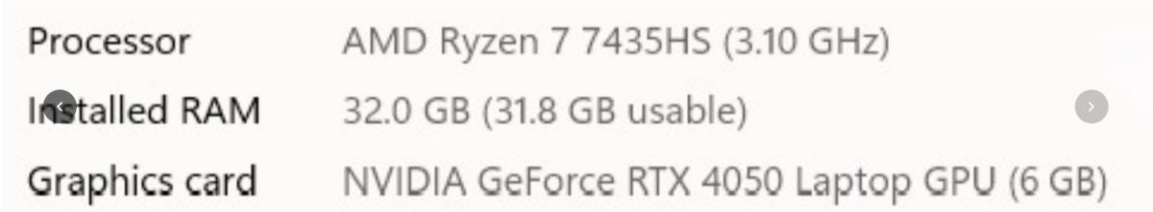
Hardware Configuration:

The experiments were conducted on a system with the following specifications:

- Processor: AMD Ryzen 7 7435HS (3.10 GHz)

- RAM: 32 GB

- GPU: NVIDIA GeForce RTX 4050 Laptop GPU (6 GB)



Software Configuration:

- CUDA Toolkit

- C/C++ with CUDA for kernel implementation

- Python (Tkinter, NumPy, PIL) for UI and benchmarking

- CMake build system

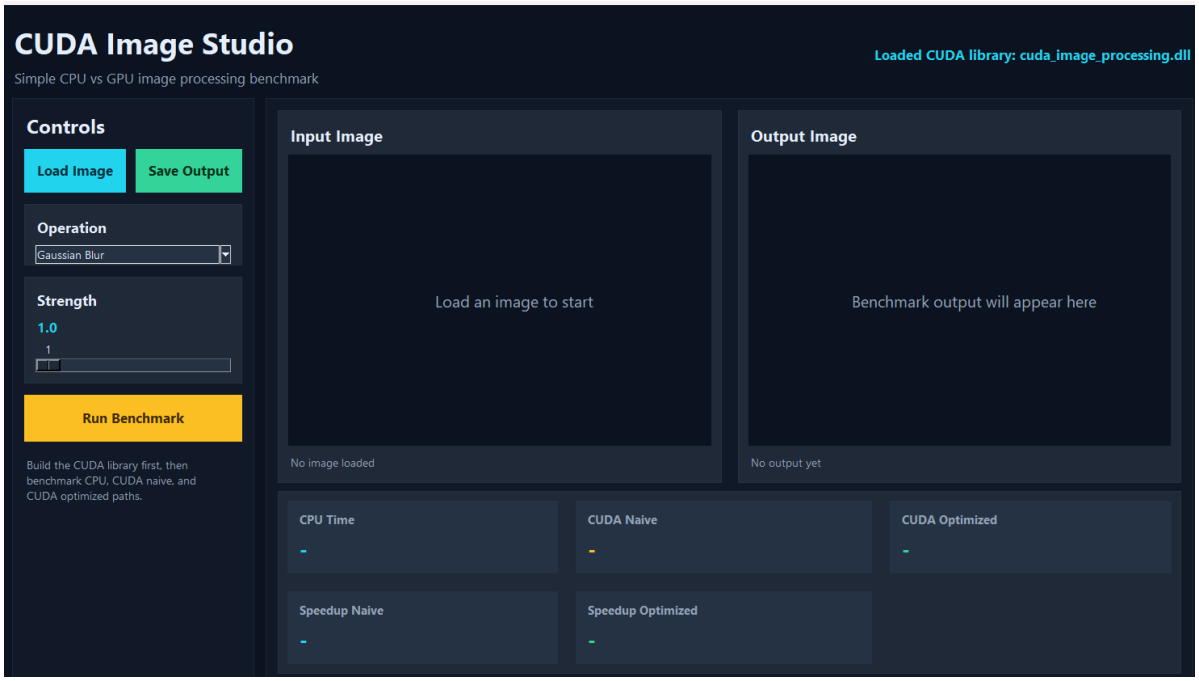


Figure 1. Application home screen

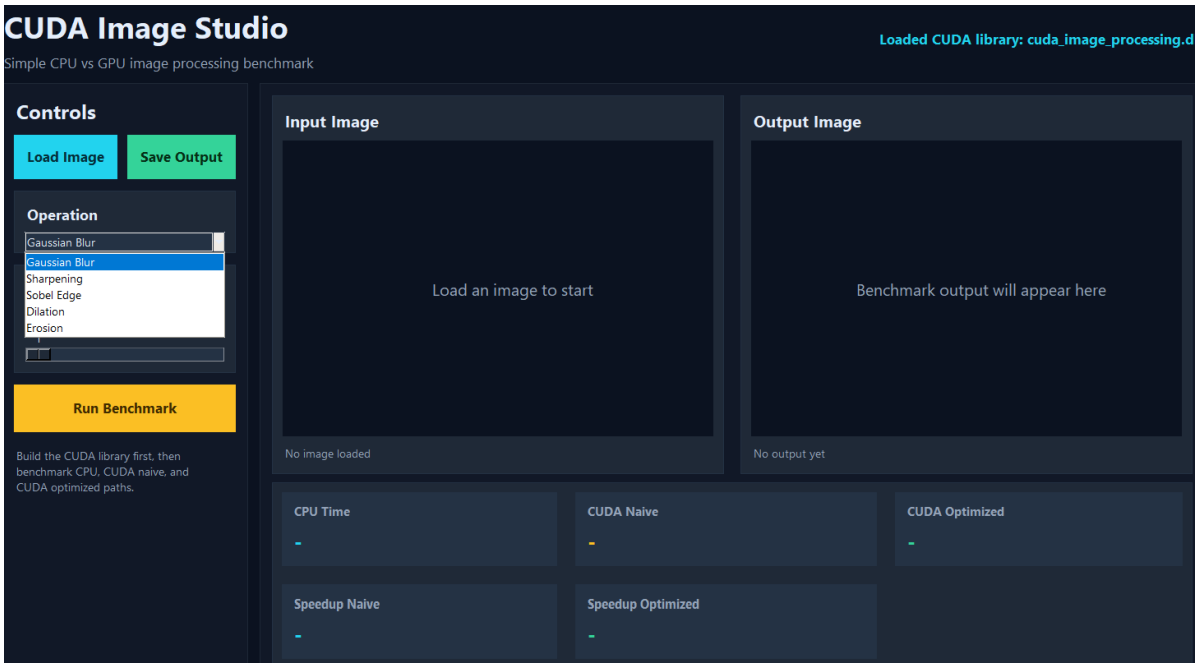


Figure 2. Operation selection interface

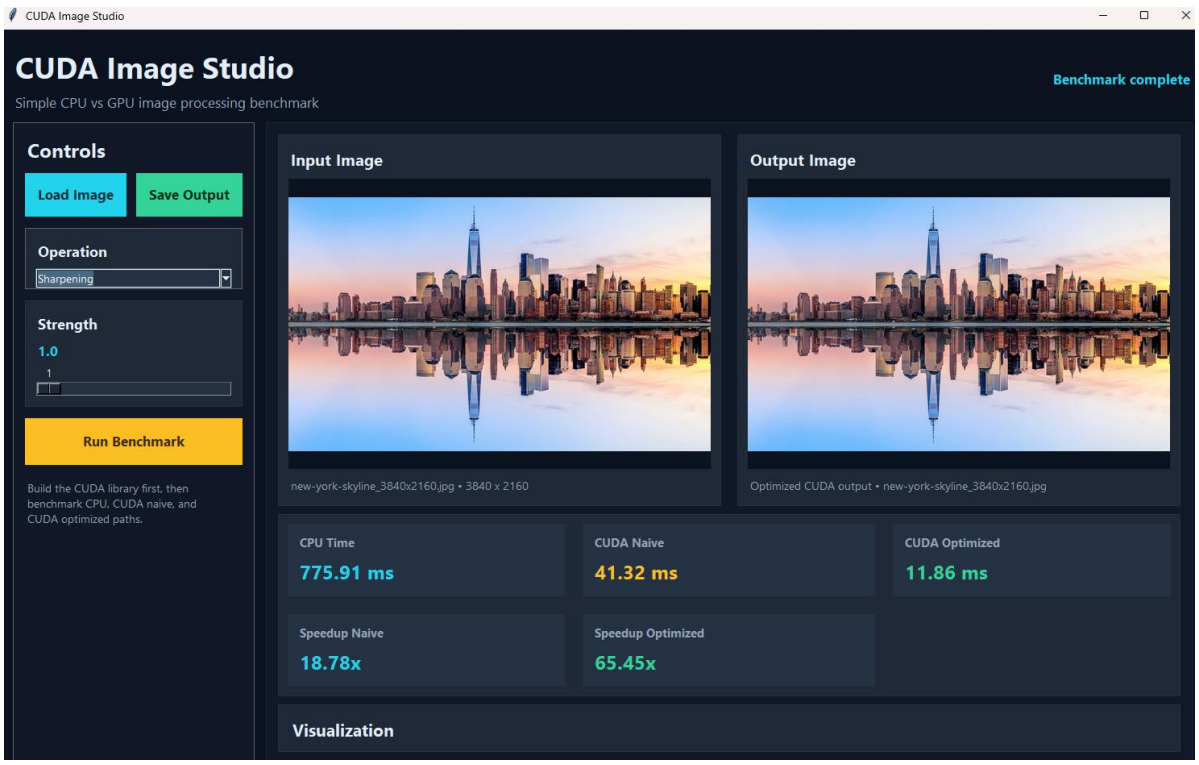


Figure 3. Sharpening benchmark results

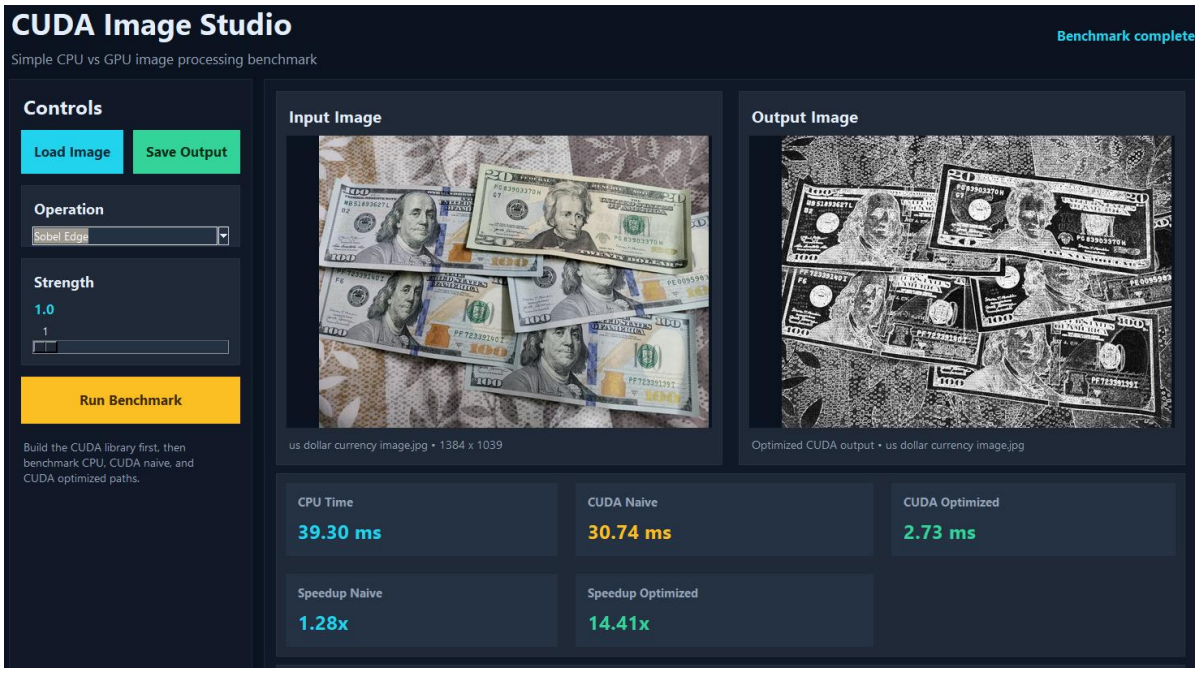


Figure 4. Sobel edge benchmark results

Figure 5. Sample input image (high-resolution skyline)

Figure 6. Sample input image (currency scene)

7. Results and Analysis

Table 1 summarizes benchmark values captured from the application screenshots. Optimized kernels consistently outperform naive kernels due to improved memory reuse and reduced global memory access overhead.

Operation	CPU Time (ms)	CUDA Naive (ms)	CUDA Optimized (ms)	Speedup Naive	Speedup Optimized
Sharpening	775.91	41.32	11.86	18.78x	65.45x
Sobel Edge	39.30	30.74	2.73	1.28x	14.41x

Interpretation: The speedup gain is operation-dependent. For high arithmetic intensity and repeated neighborhood access, shared-memory kernels provide substantial acceleration. For lighter kernels or smaller effective workloads, naive GPU gains may be moderate, while optimized kernels still provide strong improvements.

The performance improvement of optimized CUDA kernels over naive implementations is primarily due to efficient memory utilization. Naive kernels repeatedly access global memory for neighboring pixel values, resulting in high latency.

In contrast, optimized kernels use shared memory to load a tile of image data along with its surrounding halo region. This allows multiple threads within a block to reuse data efficiently, significantly reducing global memory access and improving execution speed.

Additionally, optimized kernel configurations use appropriate block and grid dimensions to maximize parallelism and GPU utilization.

8. Main CUDA Kernel Code Listings

The following listings provide the primary CUDA kernels used for optimized execution of each technique.

Gaussian Blur (Optimized Shared Memory Kernel)

```
__global__ void blur_shared_kernel(const unsigned char* input, unsigned char* output, int width,
int height) {
    constexpr int tile_width = cudaimg::kTileSize + 2;
    constexpr int tile_height = cudaimg::kTileSize + 2;
    __shared__ unsigned char tile[tile_width * tile_height * cudaimg::kChannels];

    const int block_x = blockIdx.x * cudaimg::kTileSize;
    const int block_y = blockIdx.y * cudaimg::kTileSize;
    const int thread_index = threadIdx.y * blockDim.x + threadIdx.x;
    const int threads = blockDim.x * blockDim.y;

    for (int index = thread_index; index < tile_width * tile_height; index += threads) {
        const int local_x = index % tile_width;
        const int local_y = index / tile_width;
        const int global_x = cudaimg::clamp_int(block_x + local_x - 1, 0, width - 1);
        const int global_y = cudaimg::clamp_int(block_y + local_y - 1, 0, height - 1);
        const int source = cudaimg::pixel_offset(global_x, global_y, width);
        const int target = (local_y * tile_width + local_x) * cudaimg::kChannels;
        tile[target + 0] = input[source + 0];
        tile[target + 1] = input[source + 1];
        tile[target + 2] = input[source + 2];
        tile[target + 3] = input[source + 3];
    }

    __syncthreads();

    const int x = block_x + threadIdx.x;
    const int y = block_y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    float red = 0.0f;
    float green = 0.0f;
    float blue = 0.0f;
    const int local_x = threadIdx.x + 1;
    const int local_y = threadIdx.y + 1;
```

```
    for (int kernel_y = -1; kernel_y <= 1; ++kernel_y) {
        for (int kernel_x = -1; kernel_x <= 1; ++kernel_x) {
            const int tile_index = ((local_y + kernel_y) * tile_width + (local_x + kernel_x)) *
cudaimg::kChannels;
            const float weight = kKernelDevice[(kernel_y + 1) * 3 + (kernel_x + 1)];
            red += weight * tile[tile_index + 0];
            green += weight * tile[tile_index + 1];
            blue += weight * tile[tile_index + 2];
        }
    }

    const int output_index = cudaimg::pixel_offset(x, y, width);
    output[output_index + 0] = cudaimg::clamp_byte(red / 16.0f);
    output[output_index + 1] = cudaimg::clamp_byte(green / 16.0f);
    output[output_index + 2] = cudaimg::clamp_byte(blue / 16.0f);
    output[output_index + 3] = tile[((local_y)*tile_width + local_x) * cudaimg::kChannels + 3];
}
```

Sharpening (Optimized Shared Memory Kernel)

```
__global__ void sharpen_shared_kernel(const unsigned char* input, unsigned char* output, int
width, int height, float amount) {
    constexpr int tile_width = cudaimg::kTileSize + 2;
    constexpr int tile_height = cudaimg::kTileSize + 2;
    __shared__ unsigned char tile[tile_width * tile_height * cudaimg::kChannels];

    const int block_x = blockIdx.x * cudaimg::kTileSize;
    const int block_y = blockIdx.y * cudaimg::kTileSize;
    const int thread_index = threadIdx.y * blockDim.x + threadIdx.x;
    const int threads = blockDim.x * blockDim.y;

    for (int index = thread_index; index < tile_width * tile_height; index += threads) {
        const int local_x = index % tile_width;
        const int local_y = index / tile_width;
        const int global_x = cudaimg::clamp_int(block_x + local_x - 1, 0, width - 1);
        const int global_y = cudaimg::clamp_int(block_y + local_y - 1, 0, height - 1);
        const int source = cudaimg::pixel_offset(global_x, global_y, width);
        const int target = (local_y * tile_width + local_x) * cudaimg::kChannels;
        tile[target + 0] = input[source + 0];
        tile[target + 1] = input[source + 1];
        tile[target + 2] = input[source + 2];
        tile[target + 3] = input[source + 3];
    }

    __syncthreads();

    const int x = block_x + threadIdx.x;
    const int y = block_y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    float blurred_red = 0.0f;
    float blurred_green = 0.0f;
    float blurred_blue = 0.0f;
    const int local_x = threadIdx.x + 1;
    const int local_y = threadIdx.y + 1;

    for (int kernel_y = -1; kernel_y <= 1; ++kernel_y) {
```

```
        for (int kernel_x = -1; kernel_x <= 1; ++kernel_x) {
            const int tile_index = ((local_y + kernel_y) * tile_width + (local_x + kernel_x)) *
cudaimg::kChannels;
            const float weight = kKernelDevice[(kernel_y + 1) * 3 + (kernel_x + 1)];
            blurred_red += weight * tile[tile_index + 0];
            blurred_green += weight * tile[tile_index + 1];
            blurred_blue += weight * tile[tile_index + 2];
        }
    }

    blurred_red /= 16.0f;
    blurred_green /= 16.0f;
    blurred_blue /= 16.0f;

    const int output_index = cudaimg::pixel_offset(x, y, width);
    output[output_index + 0] = cudaimg::clamp_byte(tile[((local_y)*tile_width + local_x) *
cudaimg::kChannels + 0] + amount * (tile[((local_y)*tile_width + local_x) * cudaimg::kChannels +
0] - blurred_red));
    output[output_index + 1] = cudaimg::clamp_byte(tile[((local_y)*tile_width + local_x) *
cudaimg::kChannels + 1] + amount * (tile[((local_y)*tile_width + local_x) * cudaimg::kChannels +
1] - blurred_green));
    output[output_index + 2] = cudaimg::clamp_byte(tile[((local_y)*tile_width + local_x) *
cudaimg::kChannels + 2] + amount * (tile[((local_y)*tile_width + local_x) * cudaimg::kChannels +
2] - blurred_blue));
    output[output_index + 3] = tile[((local_y)*tile_width + local_x) * cudaimg::kChannels + 3];
}
```

Sobel Edge Detection (Optimized Shared Memory Kernel)

```
__global__ void sobel_shared_kernel(const unsigned char* input, unsigned char* output, int width,
int height, float strength) {
    constexpr int tile_width = cudaimg::kTileSize + 2;
    constexpr int tile_height = cudaimg::kTileSize + 2;
    __shared__ float tile[tile_width * tile_height];

    const int block_x = blockIdx.x * cudaimg::kTileSize;
    const int block_y = blockIdx.y * cudaimg::kTileSize;
    const int thread_index = threadIdx.y * blockDim.x + threadIdx.x;
    const int threads = blockDim.x * blockDim.y;

    for (int index = thread_index; index < tile_width * tile_height; index += threads) {
        const int local_x = index % tile_width;
        const int local_y = index / tile_width;
        const int global_x = cudaimg::clamp_int(block_x + local_x - 1, 0, width - 1);
        const int global_y = cudaimg::clamp_int(block_y + local_y - 1, 0, height - 1);
        tile[index] = cudaimg::grayscale_from_rgba(input, global_x, global_y, width, height);
    }

    __syncthreads();

    const int x = block_x + threadIdx.x;
    const int y = block_y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    float gx = 0.0f;
    float gy = 0.0f;
    const int local_x = threadIdx.x + 1;
    const int local_y = threadIdx.y + 1;
```

```
    for (int kernel_y = -1; kernel_y <= 1; ++kernel_y) {
        for (int kernel_x = -1; kernel_x <= 1; ++kernel_x) {
            const int tile_index = (local_y + kernel_y) * tile_width + (local_x + kernel_x);
            const int kernel_index = (kernel_y + 1) * 3 + (kernel_x + 1);
            gx += tile[tile_index] * kSobelXDevice[kernel_index];
            gy += tile[tile_index] * kSobelYDevice[kernel_index];
        }
    }

    const int index = cudaimg::pixel_offset(x, y, width);
    const unsigned char value = cudaimg::clamp_byte(sqrtf(gx * gx + gy * gy) * fmaxf(1.0f,
strength) * 1.6f);
    output[index + 0] = value;
    output[index + 1] = value;
    output[index + 2] = value;
    output[index + 3] = input[index + 3];
}
```

Dilation (Shared Memory Morphology Kernel)

```
__global__ void morphology_shared_kernel(const unsigned char* input, unsigned char* output, int
width, int height, int dilate) {
    constexpr int tile_width = cudaimg::kTileSize + 2;
    constexpr int tile_height = cudaimg::kTileSize + 2;
    __shared__ float tile[tile_width * tile_height];

    const int block_x = blockIdx.x * cudaimg::kTileSize;
    const int block_y = blockIdx.y * cudaimg::kTileSize;
    const int thread_index = threadIdx.y * blockDim.x + threadIdx.x;
    const int threads = blockDim.x * blockDim.y;

    for (int index = thread_index; index < tile_width * tile_height; index += threads) {
        const int local_x = index % tile_width;
        const int local_y = index / tile_width;
        const int global_x = cudaimg::clamp_int(block_x + local_x - 1, 0, width - 1);
        const int global_y = cudaimg::clamp_int(block_y + local_y - 1, 0, height - 1);
        tile[index] = cudaimg::grayscale_from_rgba(input, global_x, global_y, width, height);
    }

    __syncthreads();

    const int x = block_x + threadIdx.x;
    const int y = block_y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    float extreme = dilate ? 0.0f : 255.0f;
    const int local_x = threadIdx.x + 1;
    const int local_y = threadIdx.y + 1;

    for (int kernel_y = -1; kernel_y <= 1; ++kernel_y) {
        for (int kernel_x = -1; kernel_x <= 1; ++kernel_x) {
            const int tile_index = (local_y + kernel_y) * tile_width + (local_x + kernel_x);
            extreme = dilate ? fmaxf(extreme, tile[tile_index]) : fminf(extreme,
tile[tile_index]);
        }
    }
```

```
    const int index = cudaimg::pixel_offset(x, y, width);
    const unsigned char value = cudaimg::clamp_byte(extreme);
    output[index + 0] = value;
    output[index + 1] = value;
    output[index + 2] = value;
    output[index + 3] = input[index + 3];
}
```

Erosion (Shared Memory Morphology Kernel)

```
__global__ void morphology_shared_kernel(const unsigned char* input, unsigned char* output, int
width, int height, int dilate) {
    constexpr int tile_width = cudaimg::kTileSize + 2;
    constexpr int tile_height = cudaimg::kTileSize + 2;
    __shared__ float tile[tile_width * tile_height];

    const int block_x = blockIdx.x * cudaimg::kTileSize;
    const int block_y = blockIdx.y * cudaimg::kTileSize;
    const int thread_index = threadIdx.y * blockDim.x + threadIdx.x;
    const int threads = blockDim.x * blockDim.y;

    for (int index = thread_index; index < tile_width * tile_height; index += threads) {
        const int local_x = index % tile_width;
        const int local_y = index / tile_width;
        const int global_x = cudaimg::clamp_int(block_x + local_x - 1, 0, width - 1);
        const int global_y = cudaimg::clamp_int(block_y + local_y - 1, 0, height - 1);
        tile[index] = cudaimg::grayscale_from_rgba(input, global_x, global_y, width, height);
    }

    __syncthreads();

    const int x = block_x + threadIdx.x;
    const int y = block_y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    float extreme = dilate ? 0.0f : 255.0f;
    const int local_x = threadIdx.x + 1;
    const int local_y = threadIdx.y + 1;

    for (int kernel_y = -1; kernel_y <= 1; ++kernel_y) {
        for (int kernel_x = -1; kernel_x <= 1; ++kernel_x) {
            const int tile_index = (local_y + kernel_y) * tile_width + (local_x + kernel_x);
            extreme = dilate ? fmaxf(extreme, tile[tile_index]) : fminf(extreme,
tile[tile_index]);
        }
    }

    const int index = cudaimg::pixel_offset(x, y, width);
    const unsigned char value = cudaimg::clamp_byte(extreme);
    output[index + 0] = value;
    output[index + 1] = value;
    output[index + 2] = value;
    output[index + 3] = input[index + 3];
}
```

9. Conclusions and Future Work

The project demonstrates that CUDA acceleration significantly improves image processing performance compared to CPU baselines, especially when kernels are optimized using shared-memory tiling. Future work includes wider benchmark coverage, asynchronous stream-based overlap of transfer and compute, support for larger filter kernels, and automated profiling-guided optimization.

10. References

[1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 3rd ed., Pearson, 2008.

[2] D. B. Kirk and W.-m. W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, Elsevier, 2013.

[3] NVIDIA, CUDA Toolkit Documentation. [Online]. Available: https://developer.nvidia.com/cuda-zone

[4] NVIDIA, CUDA Samples Repository. [Online]. Available: https://github.com/NVIDIA/cuda-samples

[5] M. J. Quinn, Parallel Programming in C with MPI and OpenMP, McGraw-Hill.

[6] Y. Gupta, S. Arya, A. Verma, "CUDA Image Processing Mini Project," GitHub Repository. [Online].

Available: https://github.com/yashvardhancse/pcap-cuda-image-processing-mini-project/